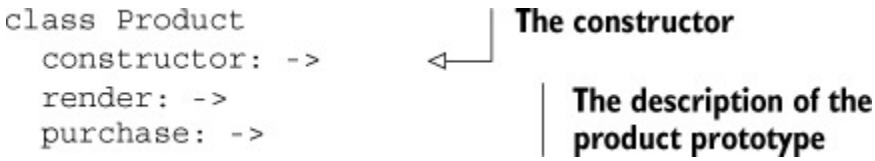


You have two things: a prototypical product and a function that constructs product objects. A class declaration achieves the same thing in a different way:



How does `extends` fit into this picture? Suppose you took the earlier `product` object (not the class) and created a new object from it:

```
camera = Object.create product
camera.megapixels = ->
```

Now if you use `construct` on this `camera` object, you can create other `camera` objects:

```
x11 = construct camera, '', stock: 6
```

This `x11` object inherits from `camera`, which in turn inherits from `product`. The `extends` keyword in a class declaration works in much the same way. It makes the prototype of one class inherit from the prototype for another class. The way that an object inherits from a prototype object that in turn inherits from another prototype is commonly referred to as the *prototype chain*.

In listing 5.3 the client application from listing 5.2 has been enhanced with an inheritance hierarchy using `extends`. To create objects from different classes, the `Shop` constructor has been modified to look at the data source and to `new` the appropriate class. Parts of the full program that are identical to listing 5.2 have been omitted from this listing.